📘 Object Oriented Systems (OOS) — CO1 Complete Detailed Solutions

🚀 Question 1 (a), (b), and (c)

Based on the examination paper provided in the uploaded file.

✨ Introduction

Object Oriented Systems (OOS) and Java Programming form the foundation of modern software engineering. Java is a powerful, platform-independent, object-oriented programming language designed with concepts such as encapsulation, inheritance, polymorphism, abstraction, multithreading, and exception handling.

In this answer, every subpart of Question 1 is explained in a highly detailed university-exam style with:

- ✅ Deep conceptual explanations
- ✅ JVM behavior discussion
- ✅ Runtime understanding
- ✅ Proper Java code examples
- ✅ Output explanation
- ✅ Conceptual analysis
- ✅ Real-world intuition

The goal is not only to answer the examination questions but also to build a strong conceptual understanding of Java and Object-Oriented Systems. 🧿 🔥

🧠 Question 1(a)

📘 Exact Question

> Fill up the blanks with appropriate phrases. Hence justify the validity of each completed sentence with proper arguments. Provide code snippets where necessary.

✨ 1(a)(i)

📘 **Statement** The _____ block is executed every time an object is created. However, the _____ block is executed only once when the _____.

✅ **Answer** The **instance initialization** block is executed every time an object is created. However, the **static initialization** block is executed only once when the **class is loaded into JVM memory**.

📘 **Detailed Explanation** In Java, initialization blocks are special blocks of code used to initialize objects and classes. Java provides two important types of initialization blocks:

Block Type	Purpose
Instance Initialization Block	Executes every time an object is created

Block Type	Purpose
Static Initialization Block	Executes only once when class loads

🚀 **Instance Initialization Block** An instance block is associated with object creation. Whenever a new object is created using the `new` keyword, the JVM allocates memory for that object inside the heap memory. Before the constructor executes, the instance block automatically runs. This behavior ensures that certain initialization logic is shared among all constructors.

✨ *Syntax*

```
{
    // instance block
}
```

🚀 **Static Initialization Block** A static block belongs to the class itself rather than individual objects. When the JVM loads a class into memory for the first time, the static block executes automatically. It executes only once irrespective of how many objects are created later.

✨ *Syntax*

```
static {
    // static block
}
```

💻 **Java Program**

```
class Demo {
    static {
        System.out.println("Static Block Executed");
    }

    {
        System.out.println("Instance Block Executed");
    }

    Demo() {
        System.out.println("Constructor Executed");
    }

    public static void main(String args[]) {
        Demo d1 = new Demo();
        Demo d2 = new Demo();
    }
}
```

🖥️ **Output**

```
Static Block Executed
Instance Block Executed
Constructor Executed
Instance Block Executed
Constructor Executed
```

🧠 **Why This Output Occurs (Step-by-Step JVM Execution)**

1. JVM loads the `Demo` class into memory.
2. Static block executes once.
3. Object `d1` is created: instance block executes, then constructor executes.
4. Object `d2` is created: instance block executes again, then constructor executes again.

💡 **Real-World Intuition** Imagine a university:

- The **static block** is like opening the university itself 🏫 → done only once.
- The **instance block** is like registering each student 👩‍🎓 → happens for every new student.

🎯 **Conclusion** Thus, instance blocks execute during every object creation, whereas static blocks execute only once during class loading by the JVM.

✨ 1(a)(ii)

📘 **Statement** There is no concept of pointers in Java because _____.

✅ **Answer** There is no concept of pointers in Java because **Java provides automatic memory management and uses references instead of explicit memory addresses**.

📘 **Detailed Explanation** Pointers are variables that directly store memory addresses. Languages like C and C++ provide pointer manipulation features. However, Java deliberately removed explicit pointer support.

This decision was taken to make Java:

- ✅ More secure
- ✅ More portable
- ✅ Easier to program
- ✅ Less error-prone

🚀 **Why Java Avoids Pointers** In C/C++, programmers can directly manipulate memory addresses (e.g., `int *p;`). This creates several risks:

- ⚠️ Memory corruption
- ⚠️ Illegal memory access
- ⚠️ Buffer overflow
- ⚠️ Dangling pointers
- ⚠️ Security vulnerabilities

Java avoids these issues completely.

🧠 **Java Uses References** Java objects are accessed using references. A reference behaves internally like a safe managed pointer, but programmers cannot directly manipulate addresses.

🖥️ **Java Example**

```
class Student {
    int roll = 10;

    public static void main(String args[]) {
        Student s = new Student();
        System.out.println(s.roll);
    }
}
```

🖥️ **Output**

```
10
```

🧠 **JVM Memory Behavior** When object `s` is created:

1. JVM allocates memory inside the heap.
2. `s` stores a reference to that object.
3. The actual memory address remains hidden.

This ensures memory safety. 🔥

💡 **Real-World Intuition** Think of Java references like hotel room numbers. You know the room number, but you cannot directly access the building wiring or plumbing. Java safely hides low-level memory details.

🎯 **Conclusion** Java removes explicit pointers to provide safer memory management, improved portability, and better security.

✨ 1(a)(iii)

📘 **Statement** The access specifiers of Java can be arranged in the order _____ < _____ < _____ < public.

✅ **Answer** The access specifiers can be arranged as: `private` < `default` < `protected` < `public`

📘 **Detailed Explanation** Access specifiers control the visibility and accessibility of class members. Java provides four levels of access control.

Access Modifier	Accessibility
private	Only within same class

Access Modifier	Accessibility
default	Same package
protected	Same package + subclasses
public	Everywhere

💻 **Java Example**

```
class Demo {
    private int a = 10;
    int b = 20;
    protected int c = 30;
    public int d = 40;

    void display() {
        System.out.println(a);
        System.out.println(b);
        System.out.println(c);
        System.out.println(d);
    }
}
```

🧠 **Deep Conceptual Analysis** Java uses access modifiers to implement:

- ✅ Encapsulation
- ✅ Data hiding
- ✅ Controlled access
- ✅ Security

This prevents unauthorized access to critical data.

🎯 **Conclusion** Thus, accessibility increases gradually from `private` to `public`.

✨ 1(a)(iv)

📘 **Statement** Method hiding occurs when _____. However, it never occurs if _____.

✅ **Answer** Method hiding occurs when **a static method in a child class has the same signature as a static method in the parent class**. However, it never occurs if **methods are non-static**.

📘 **Detailed Explanation** Method hiding is related to static methods. Unlike method overriding, static methods belong to classes, not objects. Therefore, the JVM resolves them during compile time.

💻 **Java Example**

```
class Parent {
    static void show() {
        System.out.println("Parent Static Method");
```

```
    }
}

class Child extends Parent {
    static void show() {
        System.out.println("Child Static Method");
    }
}

class Test {
    public static void main(String args[]) {
        Parent p = new Child();
        p.show();
    }
}
```

🖥️ **Output**

```
Parent Static Method
```

🧠 **Why This Output Occurs** Static methods are resolved using the reference type. Here, the reference is `Parent p`, so the JVM calls the parent version. This is method hiding, not overriding.

🎯 **Conclusion** Method hiding applies only to static methods because static binding occurs during compile time.

✨ 1(a)(v)

📘 **Statement** Arrays of objects are passed to a function by _____. Only one slot of an array of objects is passed by _____.

✅ **Answer** Arrays of objects are passed to a function by **reference**. Only one slot of an array of objects is passed by **value**.

📘 **Detailed Explanation** Java always uses pass-by-value. However, for objects and arrays, the value passed is actually the reference value. Thus:

- The entire array reference is copied.
- Both methods refer to the same underlying array.

💻 **Java Example**

```
class Demo {
    static void modify(int arr[]) {
        arr[0] = 999;
    }

    public static void main(String args[]) {
```

```
        int arr[] = {1, 2, 3};
        modify(arr);
        System.out.println(arr[0]);
    }
}
```

🖥️ **Output**

```
999
```

🧠 **JVM Understanding** Only the reference value is copied. Both references point to the same heap array object. Hence, the modification becomes visible.

🎯 **Conclusion** Java passes array references by value, allowing methods to modify the original array contents.

✨ 1(a)(vi)

📘 **Statement** The _____ methods of a Base class are _____ in Child class, but cannot be overridden.

✅ **Answer** The **final** methods of a Base class are **inherited** in Child class, but cannot be overridden.

📘 **Detailed Explanation** The `final` keyword restricts modification. When applied to methods:

- ✅ method can be inherited
- ❌ method cannot be overridden

💻 **Java Example**

```
class Parent {
    final void show() {
        System.out.println("Final Method");
    }
}

class Child extends Parent {
    // Cannot override show() here
}
```

🧠 **Why Final Methods Exist** Final methods protect critical behavior. Examples like banking security methods, authentication logic, and encryption operations should not be modified accidentally.

🎯 **Conclusion** Final methods support inheritance while preventing runtime polymorphic modification.

✨ 1(a)(vii)

📘 **Statement** All the resources used in a program are closed automatically if _____.

✅ **Answer** All the resources used in a program are closed automatically if **try-with-resources statement is used**.

📘 **Detailed Explanation** Java introduced try-with-resources in Java 7. It automatically closes resources like:

- ✅ files
- ✅ database connections
- ✅ streams
- ✅ sockets

💻 **Java Example**

```
import java.io.*;

class Demo {
    public static void main(String args[]) {
        try(FileInputStream fin = new FileInputStream("abc.txt")) {
            System.out.println("File Opened");
        } catch(Exception e) {
            System.out.println(e);
        }
    }
}
```

🧠 **JVM Behavior** After the try block finishes, the JVM automatically invokes `close()`, even if an exception occurs. This prevents memory/resource leaks. 🔥

🎯 **Conclusion** Try-with-resources simplifies resource management and improves application reliability.

🚀 Question 1(b)

📘 **Exact Question**

> State whether each can be done or not. Justify properly.

✨ 1(b)(i)

📘 **Keeping no abstract methods within an abstract class.** ✅ **Answer:** Yes, it can be done.

📘 **Detailed Explanation** An abstract class does not necessarily require abstract methods. A class may be declared abstract simply to prevent object creation.

💻 **Example**

```
abstract class Vehicle {
    void start() {
        System.out.println("Vehicle Started");
```

```
    }
}
```

🧠 **Conceptual Analysis** Even though all methods are concrete, object creation is restricted. This supports partial abstraction.

🎯 **Conclusion** An abstract class may contain zero abstract methods.

✨ 1(b)(ii)

📘 **Giving a stricter access specifier to overridden method.** ✅ **Answer:** No, it cannot be done.

📘 **Explanation** Overridden methods cannot reduce accessibility. Otherwise, polymorphism would break.

💻 **Example**

```
class Parent {
    public void show() { }
}

class Child extends Parent {
    // ERROR: Cannot reduce visibility
    // private void show() { }
}
```

🧠 **Reason** A parent reference must be able to access the child method safely. Reducing access violates inheritance principles.

🎯 **Conclusion** Access levels can increase, but cannot decrease.

✨ 1(b)(iii)

📘 **Overloading the main() method.** ✅ **Answer:** Yes, it can be done.

💻 **Example**

```
class Demo {
    public static void main(String args[]) {
        main(10);
    }

    static void main(int x) {
        System.out.println(x);
    }
}
```

🖥️ **Output**

```
10
```

🧠 **JVM Understanding** The JVM specifically calls only `public static void main(String args[])`. Other versions behave like normal overloaded methods.

🎯 **Conclusion** Main method overloading is perfectly allowed.

✨ 1(b)(iv)

📘 **Creating 2D array with rows having different columns.** ✅ **Answer:** Yes, it can be done.

📘 **Explanation** Java supports "jagged" arrays. Each row may have a different length.

🖥️ **Example**

```
class Demo {
    public static void main(String args[]) {
        int arr[][] = {
            {1, 2},
            {3, 4, 5},
            {6}
        };
        System.out.println(arr[1][2]);
    }
}
```

🖥️ **Output**

```
5
```

🎯 **Conclusion** Java allows irregular multidimensional arrays.

✨ 1(b)(v)

📘 **Putting try-catch-finally block inside finally block.** ✅ **Answer:** Yes, it can be done.

🖥️ **Example**

```
class Demo {
    public static void main(String args[]) {
        try {
            int x = 10 / 0;
```

```
        } catch(Exception e) {
            System.out.println("Exception");
        } finally {
            try {
                System.out.println("Inside Finally");
            } catch(Exception e) { }
        }
    }
}
```

🎯 **Conclusion** Nested exception handling inside a finally block is completely legal.

✨ 1(b)(vi)

📘 **Initializing non-static variables inside static block.** ✅ **Answer:** No, directly it cannot be done.

🧠 **Explanation** Static blocks belong to the class level, while non-static variables belong to objects. Since an object is absent during class loading, direct access becomes impossible.

💻 **Example**

```
class Demo {
    int x = 10;
    static {
        // ERROR
        // x = 20;
    }
}
```

🎯 **Conclusion** A static context cannot directly access instance members.

✨ 1(b)(vii)

📘 **Calling member methods in cascaded fashion by anonymous object.** ✅ **Answer:** Yes, it can be done.

💻 **Example**

```
class Demo {
    Demo show() {
        System.out.println("Show");
        return this;
    }

    void display() {
        System.out.println("Display");
    }

    public static void main(String args[]) {
```

```
        new Demo().show().display();
    }
}
```

🖥️ **Output**

```
Show
Display
```

🧠 **Explanation** Returning `this` provides the current object reference, enabling method chaining.

🎯 **Conclusion** Anonymous objects fully support cascaded method calls.

✨ 1(b)(viii)

📘 **Resolving name conflicts of classes in different packages.** ✅ **Answer:** Yes, it can be done.

📘 **Explanation** Fully qualified names are used to resolve conflicts.

💻 **Example**

```
java.util.Date d1;
java.sql.Date d2;
```

🎯 **Conclusion** Package names uniquely identify classes and prevent conflicts.

🚀 Question 1(c)

✨ 1(c)(i)

📘 **Distinguish between Reference Data Types and Primitive Data Types**

📊 **Comparison Table**

Feature	Primitive Data Types	Reference Data Types
Stores	Actual value	Memory reference
Memory Location	Stack	Heap object reference
Examples	`int`, `char`, `float`	`String`, `Array`, `Object`
Size	Fixed	Variable
Default Value	Depends on type	`null`
Methods	Not allowed	Allowed

Feature	Primitive Data Types	Reference Data Types
Performance	Faster	Slightly slower

📘 **Detailed Explanation** Primitive types directly store data values. Reference types store references pointing to heap objects.

💻 **Example**

```
class Demo {
    public static void main(String args[]) {
        int x = 10;
        String s = "Hello";

        System.out.println(x);
        System.out.println(s);
    }
}
```

🖥️ **Output**

```
10
Hello
```

🎯 **Conclusion** Primitive types improve performance while reference types enable object-oriented programming.

✨ 1(c)(ii)

📘 **Distinguish between Static Binding and Dynamic Binding**

📊 **Comparison Table**

Feature	Static Binding	Dynamic Binding
Also Called	Early Binding	Late Binding
Occurs At	Compile Time	Runtime
Used For	`static`/`final`/`private` methods	Overridden methods
Polymorphism	Not supported	Supported
Performance	Faster	Slightly slower

📘 **Detailed Explanation** Binding means connecting method calls with method bodies.

🚀 **Static Binding** Method resolution occurs during compilation. *Examples:* static methods, final methods, private methods.

🚀 **Dynamic Binding** Method resolution occurs during runtime using the actual object type. It supports runtime polymorphism.

💻 **Example**

```
class Parent {
    void show() {
        System.out.println("Parent");
    }
}

class Child extends Parent {
    void show() {
        System.out.println("Child");
    }
}

class Demo {
    public static void main(String args[]) {
        Parent p = new Child();
        p.show();
    }
}
```

🖥️ **Output**

```
Child
```

🧠 **JVM Runtime Analysis** At runtime, the JVM checks the actual object (`new Child()`). Hence, the child version executes. This demonstrates dynamic binding.

💡 **Real-World Intuition** Dynamic binding is like calling customer care. The response depends on who *actually* answers the phone.

🎯 **Final Conclusion** Static binding improves efficiency whereas dynamic binding enables runtime polymorphism and flexibility in object-oriented systems.